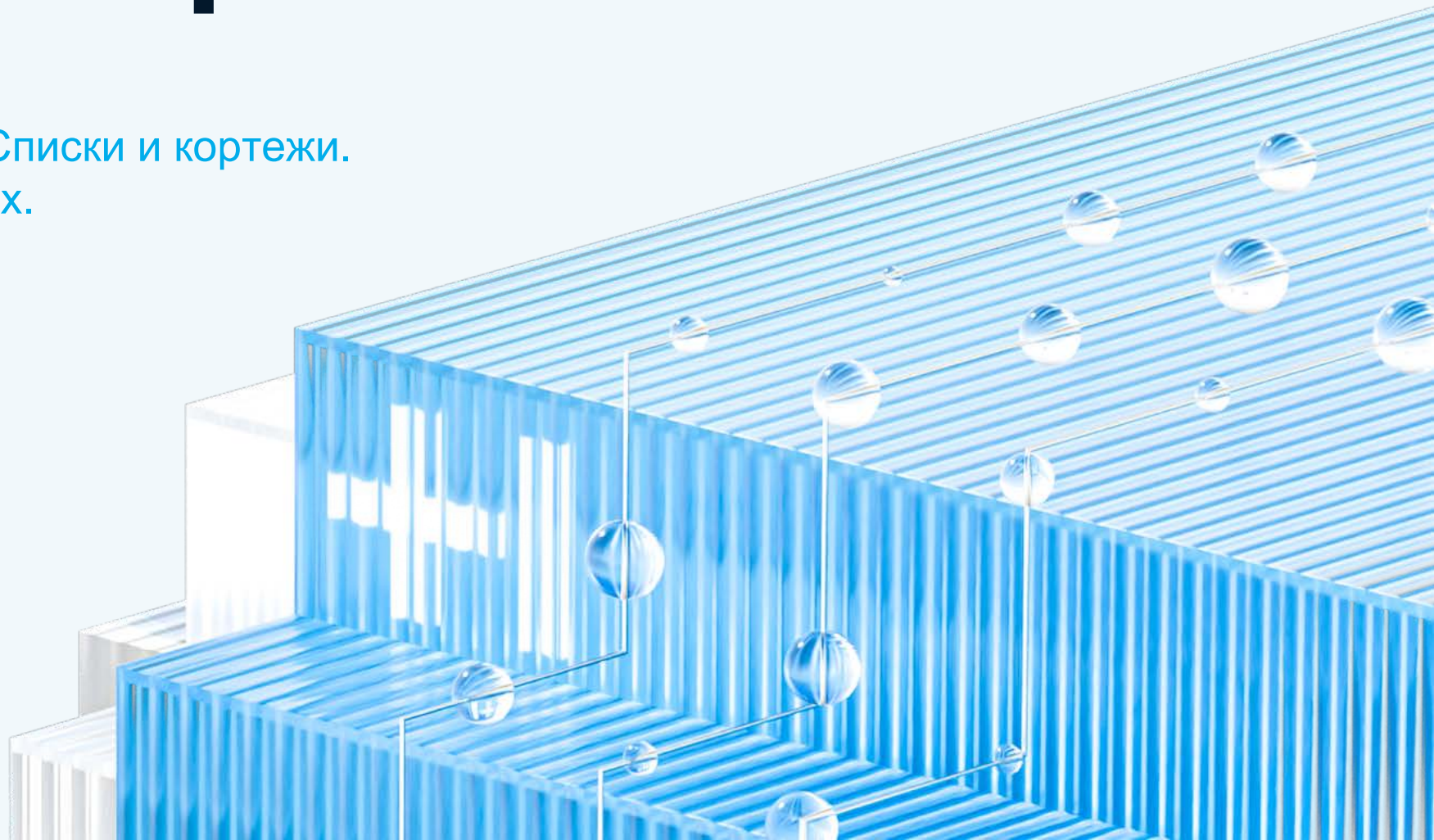
Основы программирования

22.10.2026

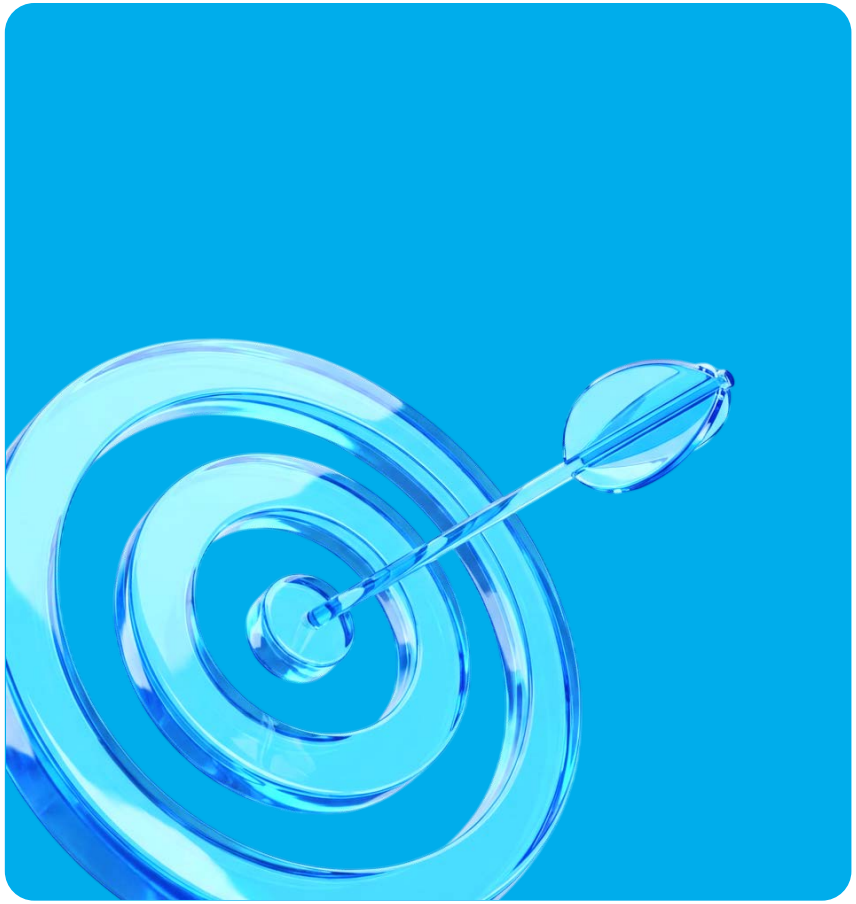
Лабораторная работа № 6. Списки и кортежи.
Обработка коллекций данных.



Проклов Роман Сергеевич

ITI

Цели лабораторной работы



1. **Познакомиться** с понятием коллекции и её видами в Python и C++
2. **Научиться** создавать списки, кортежи и массивы, выполнять базовые операции
3. **Изучить** методы и функции для работы с коллекциями
4. **Освоить** срезы, поиск, сортировку, фильтрацию коллекций
5. **Применить** полученные знания для обработки наборов данных
6. **Отработать** навыки работы с Git: коммиты, ветвление, push

Формируемые компетенции: ППК-Р1.1, ППК-Р1.4, ППК-Р1.5

План лабораторной работы

Часть 1. Теория: коллекции в Python

1. Что такое коллекция? Виды коллекций
2. Списки (list): создание, операции, методы
3. Срезы списков
4. Кортежи (tuple): создание, особенности

Часть 2. Теория: коллекции в C++

5. Массивы (array): создание, операции
6. std::vector: создание, операции, методы
7. std::array: современный массив
8. std::tuple: кортежи в C++

Часть 3. Теория: обработка коллекций

9. Поиск, сортировка, фильтрация
10. Многомерные коллекции

Часть 4. Практика

11. Задания на закрепление
12. Работа в мини-группах
13. Сдача отчёта и защита



Коллекции в Python

Коллекции: хранение наборов данных

Коллекция — это структура данных, которая позволяет хранить несколько значений в одной переменной.

Зачем нужны коллекции?

- Хранить список студентов группы
- Обрабатывать результаты измерений
- Работать с матрицами и таблицами
- Хранить наборы однотипных данных

Пример из жизни:

- Список покупок — коллекция
- Расписание занятий — коллекция
- Таблица оценок — коллекция

Виды коллекций в Python:

Коллекция	Изменяемость	Упорядоченность	Дубликаты	Пример
list	Изменяемая	Да	Да	[1, 2, 3]
tuple	Неизменяемая	Да	Да	(1, 2, 3)
dict	Изменяемая	Да (с 3.7)	Ключи уникальны	{"a": 1}
set	Изменяемая	Нет	Нет	{1, 2, 3}

Виды коллекций в C++:

Коллекция	Размер	Изменяемость	Пример
int arr[N]	Фиксированный	Изменяемая	int a[5]
std::array	Фиксированный	Изменяемая	std::array<int, 5>
std::vector	Динамический	Изменяемая	std::vector<int>
std::tuple	Фиксированный	Изменяемая	std::tuple<int, string>

Списки в Python: создание

Список (list) — это упорядоченная изменяемая коллекция элементов.

Создание списка:

```
# Пустой список
empty = []
empty2 = list()

# Список чисел
numbers = [1, 2, 3, 4, 5]

# Список строк fruits = ["яблоко", "банан", "вишня"]

# Смешанный список (разные типы)
mixed = [1, "два", 3.14, True]

# Список из range
squares = list(range(1, 6))          # [1, 2, 3, 4, 5]

# Генератор списка (list comprehension)
squares = [x ** 2 for x in range(1, 6)]          # [1, 4, 9, 16, 25]
evens = [x for x in range(10) if x % 2 == 0]     # [0, 2, 4, 6, 8]
```

Доступ к элементам:

```
fruits = ["яблоко", "банан", "вишня"]

print(fruits[0])          # яблоко (первый)
print(fruits[1])          # банан
print(fruits[-1])         # вишня (последний)
print(fruits[-2])         # банан (предпоследний)

# Длина списка
print(len(fruits))        # 3
```

Индексация:

- Положительные индексы: 0, 1, 2, ... (с начала)
- Отрицательные индексы: -1, -2, -3, ... (с конца)

Списки в Python: операции и методы

Основные операции:

```
fruits = ["яблоко", "банан", "вишня"]

# Добавление
fruits.append("груша")  # в конец
fruits.insert(1, "апельсин")          # по индексу
fruits.extend(["слива", "персик"])    # несколько элементов

# Удаление
fruits.remove("банан")  # по значению
del fruits[0]                         # по индексу
popped = fruits.pop()                 # удалить и вернуть последний
popped2 = fruits.pop(0)               # удалить и вернуть по индексу

# Изменение
fruits[0] = "киви"                    # замена элемента

# Поиск
index = fruits.index("вишня")         # индекс элемента
count = fruits.count("яблоко")        # количество вхождений

# Сортировка
numbers = [3, 1, 4, 1, 5, 9, 2, 6]
numbers.sort()                        # сортировка на месте
numbers.sort(reverse=True)            # по убыванию
sorted_copy = sorted(numbers)         # возвращает новый список
numbers.reverse()                     # обратный порядок
```

Проверка наличия:

```
if "яблоко" in fruits:
    print("Яблоко есть в списке")

if "арбуз" not in fruits:
    print("Арбуза нет в списке")
```

Объединение и повторение:

```
list1 = [1, 2, 3]
list2 = [4, 5, 6]

combined = list1 + list2              # [1, 2, 3, 4, 5, 6]
repeated = list1 * 3                  # [1, 2, 3, 1, 2, 3, 1, 2, 3]
```

Срезы списков в Python

Синтаксис среза:

```
список[начало:конец:шаг]
```

Важно:

- Конец среза **не включается**
- [::-1] — идиоматический способ реверса
- Срез создаёт **новый список** (копию)

Пример: обработка данных

```
data = [10, 20, 30, 40, 50, 60, 70, 80]

first_half = data[:len(data)//2]       # первая половина
second_half = data[len(data)//2:]  # вторая половина

print(first_half)          # [10, 20, 30, 40]
print(second_half)      # [50, 60, 70, 80]
```

Примеры:

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

# Базовые срезы
print(numbers[2:5])     # [2, 3, 4] (с 2 по 4, не включая 5)
print(numbers[:3])      # [0, 1, 2] (с начала до 2)
print(numbers[5:])      # [5, 6, 7, 8, 9] (с 5 до конца)
print(numbers[:])       # [0, 1, 2, ..., 9] (копия всего списка)

# С шагом
print(numbers[::2])     # [0, 2, 4, 6, 8] (каждый второй)
print(numbers[1::2])    # [1, 3, 5, 7, 9] (каждый второй, начиная с 1)
print(numbers[::-1])    # [9, 8, 7, ..., 0] (реверс)

# Отрицательные индексы
print(numbers[-3:])     # [7, 8, 9] (последние три)
print(numbers[:-3])      # [0, 1, ..., 6] (всё, кроме последних трёх)

# Замена через срез
numbers[2:5] = [20, 30, 40]    # заменить элементы 2, 3, 4
```

Кортежи в Python

Кортеж (tuple) — это упорядоченная **неизменяемая** коллекция элементов.

Создание кортежа:

```
# Пустой кортеж
empty = () empty2 = tuple()

# Кортеж с элементами
point = (3, 5)
colors = ("красный", "зелёный", "синий")

# Кортеж из одного элемента (обязательна запятая!)
single = (42,)          # кортеж
not_tuple = (42)        # это просто число в скобках!

# Без скобок (упаковка)
coordinates = 10, 20, 30            # кортеж

# Из списка
tuple_from_list = tuple([1, 2, 3])    # (1, 2, 3)
```

Доступ к элементам:

```
point = (3, 5)
print(point[0])          # 3
print(point[1])          # 5
print(point[-1])         # 5

# Распаковка
x, y = point print(x, y)  # 3 5

# Множественное присваивание
a, b, c = 1, 2, 3
```

Основные операции:

```
point = (3, 5, 7, 3)

print(len(point))        # 4
print(point.count(3))    # 2
print(point.index(5))    # 1
print(5 in point)        # True

# Срезы (как у списков)
print(point[1:3])        # (5, 7)
print(point[::-1])       # (3, 7, 5, 3)
```

Что нельзя делать:

```
point = (3, 5)
# point[0] = 10          # ОШИБКА! Кортеж неизменяемый
# point.append(7)        # ОШИБКА! Нет метода append
```

Python: списки vs кортежи

Аспект	Список (list)	Кортеж (tuple)
Изменяемость	Изменяемый	Неизменяемый
Синтаксис	[1, 2, 3]	(1, 2, 3)
Методы	Много (append, remove, ...)	Мало (count, index)
Производительность	Медленнее	Быстрее
Использование как ключ словаря	Нет	Да
Распаковка	Да	Да
Срезы	Да	Да
Применение	Динамические данные	Неизменяемые данные

Когда использовать список:

- Данные будут меняться (добавление, удаление)
- Нужны методы append, remove, sort
- Динамический размер

Когда использовать кортеж:

- Данные не должны меняться
- Нужна защита от случайного изменения
- Использование как ключ словаря
- Возврат нескольких значений из функции

Пример: возврат из функции

```
def get_min_max(numbers):
    return min(numbers), max(numbers)        # возвращает кортеж

minimum, maximum = get_min_max([3, 1, 4, 1, 5])
print(f"Мин: {minimum}, Макс: {maximum}")
```

Коллекции в C++

Массивы в C++: создание

Массив — то коллекция элементов одного типа, расположенных в памяти последовательно.

Создание массива:

```cpp
#include <iostream>

int main() {
    // Статический массив фиксированного размера
    int numbers[5];     // 5 элементов (неинициализированы)
    int numbers2[5] = {1, 2, 3, 4, 5}; // полная инициализация
    int numbers3[5] = {1, 2};          // остальные = 0
    int numbers4[] = {1, 2, 3, 4, 5};  // размер выводится (5)

    // Массив строк
    std::string fruits[3] = {"яблоко", "банан", "вишня"};

    // Доступ к элементам
    std::cout << numbers2[0] << "\n";    // 1
    std::cout << numbers2[4] << "\n";    // 5

    // Изменение
    numbers2[0] = 10;

    return 0;
}
```

Важно:

- Размер массива **фиксирован** и известен на этапе компиляции
- Индексация с 0
- **Нет проверки границ!** numbers[10] для массива из 5 — неопределённое поведение

Размер массива:

```cpp
int numbers[] = {1, 2, 3, 4, 5};
int size = sizeof(numbers) / sizeof(numbers[0]);          // 5
```

Размер массива:

```cpp
int matrix[3][4];          // 3 строки, 4 столбца
int matrix2[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};

matrix2[0][1] = 20;      // доступ к элементу
```

std::vector в C++: динамический массив

std::vector — то динамический массив из стандартной библиотеки. Может изменять размер во время выполнения.

Создание:

```cpp
#include <iostream>
#include <vector>

int main() {
    // Пустой вектор
    std::vector<int> empty;

    // Вектор с элементами
    std::vector<int> numbers = {1, 2, 3, 4, 5};

    // Вектор из N элементов
    std::vector<int> zeros(5);          // {0, 0, 0, 0, 0}
    std::vector<int> tens(5, 10);       // {10, 10, 10, 10, 10}

    // Вектор строк
    std::vector<std::string> fruits = {"яблоко", "банан"};

    // Доступ
    std::cout << numbers[0] << "\n";    // 1
    std::cout << numbers.at(1) << "\n"; // 2 (с проверкой границ)
    std::cout << numbers.front() << "\n";  // 1
    std::cout << numbers.back() << "\n";  // 5

    return 0;
}
```

Основные операции:

```cpp
std::vector<int> v = {1, 2, 3};

// Добавление
v.push_back(4);             // {1, 2, 3, 4}
v.insert(v.begin(), 0);     // {0, 1, 2, 3, 4}

// Удаление
v.pop_back();               // {0, 1, 2, 3}
v.erase(v.begin());         // {1, 2, 3}

// Размер
std::cout << v.size() << "\n";          // 3
std::cout << v.empty() << "\n";         // 0 (false)

// Очистка
v.clear();                  // пустой вектор

// Изменение размера
v.resize(10);               // {0, 0, 0, 0, 0, 0, 0, 0, 0, 0}
```

Итерация:

```cpp
// Классический for
for (size_t i = 0; i < v.size(); ++i) {
    std::cout << v[i] << " ";
}

// Range-based for (C++11)
for (int x : v) {
    std::cout << x << " ";
}

// С итераторами
for (auto it = v.begin(); it != v.end(); ++it) {
    std::cout << *it << " ";
}
```

std::array в C++: современный массив

std::array — это обёртка над обычным массивом, добавленная в C++11. Даёт безопасность и удобство std::vector, но с фиксированным размером.

Создание:

```cpp
#include <iostream>
#include <array>

int main() {
    // Создание
    std::array<int, 5> arr = {1, 2, 3, 4, 5};
    std::array<int, 3> zeros = {};          // {0, 0, 0}
    std::array<double, 4> values = {1.1, 2.2, 3.3, 4.4};

    // Доступ
    std::cout << arr[0] << "\n";            // 1
    std::cout << arr.at(1) << "\n";         // 2 (с проверкой)
    std::cout << arr.front() << "\n";       // 1
    std::cout << arr.back() << "\n";        // 5

    // Размер (известен на этапе компиляции)
    std::cout << arr.size() << "\n";        // 5

    return 0;
}
```

Преимущества над обычным массивом:

Аспект	int arr[5]	std::array<int, 5>
Размер	sizeof(arr)/sizeof(arr[0])	arr.size()
Проверка границ	Нет	.at()
Копирование	Нет	Да (присваивание)
Передача в функцию	Указатель	Копия или ссылка
Range-based for	Да	Да

Итерация:

```cpp
std::array<int, 5> arr = {1, 2, 3, 4, 5};

// Range-based for
for (int x : arr) {
    std::cout << x << " ";
}

// С индексом
for (size_t i = 0; i < arr.size(); ++i) {
    std::cout << arr[i] << " ";
}
```

Многомерный std::array:

```cpp
std::array<std::array<int, 3>, 2> matrix = {{
    {1, 2, 3},
    {4, 5, 6}
}};
```

std::tuple в C++: кортежи

std::tuple — это коллекция элементов разных типов фиксированного размера.
Аналог Python-кортежа.

Создание:

```cpp
#include <iostream>
#include <tuple>
#include <string>

int main() {
    // Создание кортежа
    std::tuple<int, std::string, double> person = {25, "Иван", 1.85};

    // С помощью std::make_tuple
    auto person2 = std::make_tuple(30, "Мария", 1.70);

    // Пустой кортеж
    std::tuple<> empty;

    return 0;
}
```

Размер кортежа:

```cpp
std::tuple<int, std::string, double> person = {25, "Иван", 1.85};
std::cout << std::tuple_size<decltype(person)>::value << "\n";    // 3
```

Доступ к элементам:

```cpp
std::tuple<int, std::string, double> person = {25, "Иван", 1.85};

// Через std::get<индекс>
std::cout << std::get<0>(person) << "\n";                // 25
std::cout << std::get<1>(person) << "\n";                // Иван
std::cout << std::get<2>(person) << "\n";                // 1.85

// Через std::get<тип> (если тип уникален)
std::cout << std::get<int>(person) << "\n";              // 25

// Изменение
std::get<0>(person) = 26;
```

Распаковка (C++17):

```cpp
#include <tuple>

auto person = std::make_tuple(25, "Иван", 1.85);

// Структурное связывание
auto [age, name, height] = person;

std::cout << "Возраст: " << age << "\n";
std::cout << "Имя: " << name << "\n";
std::cout << "Рост: " << height << "\n";
```

Применение:

- Возврат нескольких значений из функции
- Хранение разнотипных данных
- Группировка связанных значений

Python vs C++: сравнение коллекций

Python	C++	Особенности
list	std::vector	Динамический, изменяемый
tuple	std::tuple	Фиксированный, разнотипный
list	std::array / int[]	Фиксированный размер
—	std::list	Двусвязный список
dict	std::map / std::unordered_map	Ключ-значение
set	std::set / std::unordered_set	Уникальные элементы

Пример: создание и вывод

Python:

```python
# Список
numbers = [1, 2, 3, 4, 5]

# Кортеж
point = (3, 5)

# Вывод
print(numbers)
print(point)
```

C++:

```cpp
#include <iostream> #include <vector> #include <tuple>

int main() {
    // Вектор
    std::vector<int> numbers = {1, 2, 3, 4, 5};

    // Кортеж
    auto point = std::make_tuple(3, 5);

    // Вывод вектора
    for (int x : numbers) std::cout << x << " "; std::cout << "\n";

    // Вывод кортежа
    std::cout << "(" << std::get<0>(point) << ", " << std::get<1>(point) << ")\n";

    return 0;
}
```

Ключевые различия:

- Python: встроенные коллекции с богатым API
- C++: коллекции из STL, требуют подключения заголовков
- Python: динамическая типизация, C++: статическая

Обработка коллекций

+I

Поиск и фильтрация в Python

Поиск элемента:

```
numbers = [10, 20, 30, 40, 50]

# Проверка наличия
if 30 in numbers:
    print("30 есть в списке")

# Индекс элемента
index = numbers.index(30)     # 2

# Поиск с условием (генератор)
found = next((x for x in numbers if x > 25), None)
print(found)     # 30
```

Поиск min/max:

```
numbers = [10, 20, 30, 40, 50]

print(min(numbers))        # 10
print(max(numbers))        # 50

# С ключом
words = ["яблоко", "банан", "вишня"]
longest = max(words, key=len)          # яблоко (6 букв)
```

Фильтрация:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Через list comprehension
evens = [x for x in numbers if x % 2 == 0]          # [2, 4, 6, 8, 10]
odds = [x for x in numbers if x % 2 != 0]           # [1, 3, 5, 7, 9]
greater_than_5 = [x for x in numbers if x > 5]      # [6, 7, 8, 9, 10]

# Через filter
evens2 = list(filter(lambda x: x % 2 == 0, numbers))

# Условия с AND/OR
filtered = [x for x in numbers if x > 3 and x < 8]      # [4, 5, 6, 7]
```

Сумма, среднее:

```
numbers = [10, 20, 30, 40, 50]

print(sum(numbers))                 # 150
print(sum(numbers) / len(numbers))  # 30.0
```

Any и all:

```
numbers = [2, 4, 6, 8]

print(any(x > 5 for x in numbers))      # True (есть > 5)
print(all(x > 0 for x in numbers))      # True (все > 0)
```

Поиск и фильтрация в C++

Поиск элемента:

```
#include <iostream> #include <vector> #include <algorithm>

int main() {
    std::vector<int> numbers = {10, 20, 30, 40, 50};

    // Поиск через std::find
    auto it = std::find(numbers.begin(), numbers.end(), 30);
    if (it != numbers.end()) {
        std::cout << "Найдено: " << *it << "\n";
        std::cout << "Индекс: " << std::distance(numbers.begin(), it) << "\n";
    } else {
        std::cout << "Не найдено\n";
    }

    // Поиск с условием
    auto it2 = std::find_if(numbers.begin(), numbers.end(), [](int x) { return x > 25; });
    if (it2 != numbers.end()) {
        std::cout << "Первое > 25: " << *it2 << "\n";      // 30
    }

    return 0;
}
```

Поиск min/max:

```
bool any_gt_5 = std::any_of(numbers.begin(), numbers.end(),
                            [](int x) { return x > 5; });        // true

bool all_positive = std::all_of(numbers.begin(), numbers.end(),
                            [](int x) { return x > 0; });        // true
```

Фильтрация (через копирование):

```
#include <vector> #include <algorithm>

std::vector<int> numbers = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

// Только чётные
std::vector<int> evens;
std::copy_if(numbers.begin(), numbers.end(), std::back_inserter(evens), [](int x) { return x % 2 == 0; });
// evens = {2, 4, 6, 8, 10}
```

Min/max:

```
#include <algorithm>

std::vector<int> numbers = {10, 20, 30, 40, 50};

auto min_it = std::min_element(numbers.begin(), numbers.end());
auto max_it = std::max_element(numbers.begin(), numbers.end());

std::cout << "Min: " << *min_it << "\n";       // 10
std::cout << "Max: " << *max_it << "\n";       // 50
```

Сумма:

```
#include <numeric>

std::vector<int> numbers = {10, 20, 30, 40, 50};

int sum = std::accumulate(numbers.begin(), numbers.end(), 0);
std::cout << "Сумма: " << sum << "\n";                    // 150

double average = static_cast<double>(sum) / numbers.size();
std::cout << "Среднее: " << average << "\n";              // 30.0
```

Сортировка в Python

Основные способы:

```
numbers = [3, 1, 4, 1, 5, 9, 2, 6]

# sort() — сортировка на месте
numbers.sort()
print(numbers)              # [1, 1, 2, 3, 4, 5, 6, 9]

# sorted() — возвращает новый список
original = [3, 1, 4, 1, 5]
sorted_copy = sorted(original)
print(original)             # [3, 1, 4, 1, 5] (не изменён)
print(sorted_copy)          # [1, 1, 3, 4, 5]

# По убыванию
numbers.sort(reverse=True)
```

Сортировка кортежей:

```
# По первому элементу, потом по второму
students = [("Иван", 25), ("Мария", 22), ("Пётр", 25)]
students.sort()
# [('Иван', 25), ('Мария', 22), ('Пётр', 25)]

# По возрасту (второй элемент)
students.sort(key=lambda s: s[1])
# [('Мария', 22), ('Иван', 25), ('Пётр', 25)]

# По нескольким ключам
students.sort(key=lambda s: (s[1], s[0]))
```

Сортировка с ключом:

```
# По длине строки
words = ["яблоко", "банан", "киви", "вишня"]
words.sort(key=len)
print(words) # ['киви', 'банан', 'вишня', 'яблоко']

# По последней букве
words.sort(key=lambda w: w[-1]) print(words)

# По абсолютному значению
numbers = [-5, 3, -1, 4, -2]
numbers.sort(key=abs)
print(numbers)              # [-1, -2, 3, 4, -5]
```

Сортировка словаря:

```
person = {"Иван": 25, "Мария": 22, "Пётр": 30}

# По ключам
sorted_keys = sorted(person.keys())

# По значениям
sorted_items = sorted(person.items(), key=lambda x: x[1])
# [('Мария', 22), ('Иван', 25), ('Пётр', 30)]
```

Стабильность сортировки:

Python использует **Timsort** — стабильный алгоритм. Это значит, что равные элементы сохраняют относительный порядок.

Сортировка в C++

Основные способы:

```cpp
#include <iostream> #include <vector> #include <algorithm>

int main() {
    std::vector<int> numbers = {3, 1, 4, 1, 5, 9, 2, 6};

    // Сортировка по возрастанию
    std::sort(numbers.begin(), numbers.end());
    // {1, 1, 2, 3, 4, 5, 6, 9}

    // По убыванию
    std::sort(numbers.begin(), numbers.end(), std::greater<int>());

    // С лямбдой
    std::sort(numbers.begin(), numbers.end(), [](int a, int b) { return a > b; });

    // Вывод
    for (int x : numbers)
    std::cout << x << " "; std::cout << "\n";

    return 0;
}
```

Стабильность:

```cpp
// std::stable_sort — стабильная сортировка
std::stable_sort(v.begin(), v.end(), comparator);
```

Проверка отсортированности:

```cpp
bool is_sorted = std::is_sorted(v.begin(), v.end());
```

Сортировка с ключом (через лямбду):

```cpp
#include <string>

std::vector<std::string> words = {"яблоко", "банан", "киви", "вишня"};

// По длине
std::sort(words.begin(), words.end(), [](const std::string& a, const std::string& b) {
            return a.length() < b.length();
        });
// {"киви", "банан", "вишня", "яблоко"}

// По последней букве
std::sort(words.begin(), words.end(), [](const std::string& a, const std::string& b) {
            return a.back() < b.back();
        });
```

Сортировка кортежей:

```cpp
#include <tuple> #include <vector>

std::vector<std::tuple<std::string, int>> students = { {"Иван", 25}, {"Мария", 22}, {"Пётр", 30} };

// Сортировка по возрасту (второй элемент)
std::sort(students.begin(), students.end(), [](const auto& a, const auto& b) {
                                    return std::get<1>(a) < std::get<1>(b);
                });

// По имени, потом по возрасту
std::sort(students.begin(), students.end());
// tuple сравнивается лексикографически: сначала первый, потом второй
```

Многомерные коллекции в Python

Матрицы (список списков):

```
# Создание матрицы 3x3
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

# Доступ к элементам
print(matrix[0][0])          # 1
print(matrix[1][2])          # 6
print(matrix[2][1])          # 8

# Размер
rows = len(matrix)           # 3
cols = len(matrix[0])        # 3
```

Умножение матриц:

```
def matrix_multiply(A, B):
    rows_A = len(A)
    cols_A = len(A[0])
    cols_B = len(B[0])

    result = [[0] * cols_B   for _ in range(rows_A)]

    for i in range(rows_A):
        for j in range(cols_B):
            for k in range(cols_A): result[i][j] += A[i][k] * B[k][j]

    return result
```

Создание матрицы через генератор:

```
# Матрица 3x4, заполненная нулями
rows, cols = 3, 4
matrix = [[0] * cols for _ in range(rows)]

# ВНИМАНИЕ: не используйте [[0] * cols] * rows — это ссылки на один список!
```

Транспонирование:

```
matrix = [
    [1, 2, 3],
    [4, 5, 6]
]

# Через zip
transposed = [list(row) for row in zip(*matrix)]
# [[1, 4], [2, 5], [3, 6]] # Через генератор

transposed2 = [[matrix[i][j] for i in range(len(matrix))] for j in range(len(matrix[0]))]
```

Вывод матрицы:

```
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

for row in matrix:
    for elem in row:
        print(f"{elem:4}", end="")
    print()
```

Многомерные коллекции в C++

Матрицы (двумерный массив):

```cpp
#include <iostream>

int main() {
    // Двумерный массив 3x3
    int matrix[3][3] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };

    // Доступ
    std::cout << matrix[0][0] << "\n";      // 1
    std::cout << matrix[1][2] << "\n";      // 6

    // Вывод
    for (int i = 0; i < 3; ++i) {
        for (int j = 0; j < 3; ++j) {
            std::cout << matrix[i][j] << " ";
        }
        std::cout << "\n";
    }

    return 0;
}
```

Вектор векторов:

```cpp
#include <vector>

// Создание матрицы 3x4, заполненной нулями
int rows = 3, cols = 4;
std::vector<std::vector<int>> matrix(rows, std::vector<int>(cols, 0));

// Доступ
matrix[0][0] = 1;
matrix[1][2] = 5;

// Вывод
for (const auto& row : matrix) {
    for (int elem : row) {
        std::cout << elem << " ";
    }
    std::cout << "\n";
}
```

std::array для фиксированного размера:

```cpp
#include <array>

std::array<std::array<int, 3>, 3> matrix = {{
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
}};
```

Динамическое создание:

```cpp
int rows, cols;
std::cout << "Введите размеры: ";
std::cin >> rows >> cols;

std::vector<std::vector<int>> matrix(rows,
                                     std::vector<int>(cols));

// Заполнение
for (int i = 0; i < rows; ++i) {
    for (int j = 0; j < cols; ++j) {
        matrix[i][j] = i * cols + j;
    }
}
```

Транспонирование:

```cpp
std::vector<std::vector<int>> transpose(
    const std::vector<std::vector<int>>& m) {

    int rows = m.size();
    int cols = m[0].size();

    std::vector<std::vector<int>> result(cols, std::vector<int>(rows));

    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            result[j][i] = m[i][j];
        }
    }

    return result;
}
```

Практика

Задание 1: Работа со списком

Задача: Написать программу, которая:

1. Создаёт список из N случайных чисел
2. Находит сумму, среднее, минимум, максимум
3. Сортирует список
4. Удаляет дубликаты (сохраняя порядок)

```python
import random

n = int(input("Введите N: "))
numbers = [random.randint(1, 100) for _ in range(n)]

print(f"Исходный список: {numbers}")
print(f"Сумма: {sum(numbers)}")
print(f"Среднее: {sum(numbers) / len(numbers):.2f}")
print(f"Минимум: {min(numbers)}")
print(f"Максимум: {max(numbers)}")

# Сортировка
numbers.sort()
print(f"Отсортированный: {numbers}")

# Удаление дубликатов (сохраняя порядок)
unique = list(dict.fromkeys(numbers))
print(f"Без дубликатов: {unique}")
```

```cpp
#include <algorithm>
#include <numeric>
#include <random>
#include <iomanip>

int main() {
    int n;
    std::cout << "Введите N: ";
    std::cin >> n;

    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<> dist(1, 100);

    std::vector<int> numbers(n);
    for (int& x : numbers) x = dist(gen);

    std::cout << "Исходный список: ";
    for (int x : numbers) std::cout << x << " ";
    std::cout << "\n";

    int sum = std::accumulate(numbers.begin(), numbers.end(), 0);
    std::cout << "Сумма: " << sum << "\n";
    std::cout << std::fixed << std::setprecision(2);
    std::cout << "Среднее: " << static_cast<double>(sum) / n << "\n";
    std::cout << "Минимум: " << *std::min_element(numbers.begin(), numbers.end()) << "\n";
    std::cout << "Максимум: " << *std::max_element(numbers.begin(), numbers.end()) << "\n";

    std::sort(numbers.begin(), numbers.end());
    std::cout << "Отсортированный: ";
    for (int x : numbers) std::cout << x << " ";
    std::cout << "\n";

    // Удаление дубликатов
    numbers.erase(std::unique(numbers.begin(), numbers.end()), numbers.end());
    std::cout << "Без дубликатов: ";
    for (int x : numbers) std::cout << x << " ";
    std::cout << "\n";

    return 0;
}
```

Задание 2: Поиск дубликатов

Задача: Найти все дубликаты в списке.

```python
numbers = [1, 2, 3, 2, 4, 5, 1, 6, 3, 7]

# Через словарь с подсчётом
from collections import Counter
counts = Counter(numbers)
duplicates = [num for num, count in counts.items() if count > 1]

print(f"Дубликаты: {duplicates}")  # [1, 2, 3]

# Через множества
seen = set()
duplicates2 = set()
for num in numbers:
    if num in seen:
        duplicates2.add(num)
    seen.add(num)

print(f"Дубликаты: {list(duplicates2)}")
```

```cpp
#include <iostream>
#include <vector>
#include <set>
#include <map>

int main() {
    std::vector<int> numbers = {1, 2, 3, 2, 4, 5, 1, 6, 3, 7};

    // Через std::map для подсчёта
    std::map<int, int> counts;
    for (int num : numbers) {
        counts[num]++;
    }

    std::cout << "Дубликаты: ";
    for (const auto& [num, count] : counts) {
        if (count > 1) {
            std::cout << num << " ";
        }
    }
    std::cout << "\n";

    // Через std::set
    std::set<int> seen;
    std::set<int> duplicates;
    for (int num : numbers) {
        if (seen.count(num)) {
            duplicates.insert(num);
        }
        seen.insert(num);
    }

    std::cout << "Дубликаты (set): ";
    for (int num : duplicates) {
        std::cout << num << " ";
    }
    std::cout << "\n";

    return 0;
}
```

Задание 3: Транспонирование матрицы

Задача: Транспонировать матрицу
(поменять строки и столбцы местами).

```python
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

# Через zip
transposed = [list(row) for row in zip(*matrix)]

# Вывод
print("Исходная матрица:")
for row in matrix:
    print(row)

print("\nТранспонированная матрица:")
for row in transposed:
    print(row)
```

```cpp
#include <iostream>
#include <vector>

int main() {
    std::vector<std::vector<int>> matrix = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };

    int rows = matrix.size();
    int cols = matrix[0].size();

    std::vector<std::vector<int>> transposed(cols,
                                             std::vector<int>(rows));

    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            transposed[j][i] = matrix[i][j];
        }
    }

    // Вывод
    std::cout << "Исходная матрица:\n";
    for (const auto& row : matrix) {
        for (int x : row) std::cout << x << " ";
        std::cout << "\n";
    }

    std::cout << "\nТранспонированная матрица:\n";
    for (const auto& row : transposed) {
        for (int x : row) std::cout << x << " ";
        std::cout << "\n";
    }

    return 0;
}
```

Задание 4: Работа с кортежами

Задача: Создать список студентов (имя, возраст, средний балл). Найти:

- Самого старшего
- С самым высоким баллом
- Средний возраст

```python
students = [
    ("Иван", 25, 4.5),
    ("Мария", 22, 4.8),
    ("Пётр", 30, 4.2),
    ("Анна", 20, 4.9)
]

# Самый старший
oldest = max(students, key=lambda s: s[1])
print(f"Самый старший: {oldest[0]}, {oldest[1]} лет")

# С самым высоким баллом
best = max(students, key=lambda s: s[2])
print(f"Лучший студент: {best[0]}, балл {best[2]}")

# Средний возраст
avg_age = sum(s[1] for s in students) / len(students)
print(f"Средний возраст: {avg_age:.1f}")

# Сортировка по баллу (убывание)
sorted_students = sorted(students, key=lambda s: s[2], reverse=True)
print("\nПо баллу:")
for name, age, gpa in sorted_students:
    print(f"{name}: {gpa}")
```

```cpp
#include <iostream>
#include <vector>
#include <tuple>
#include <algorithm>
#include <numeric>
#include <iomanip>

int main() {
    std::vector<std::tuple<std::string, int, double>> students = {
        {"Иван", 25, 4.5},
        {"Мария", 22, 4.8},
        {"Пётр", 30, 4.2},
        {"Анна", 20, 4.9}
    };

    // Самый старший
    auto oldest = *std::max_element(students.begin(), students.end(),
        [](const auto& a, const auto& b) {
            return std::get<1>(a) < std::get<1>(b);
        });
    std::cout << "Самый старший: " << std::get<0>(oldest)
              << ", " << std::get<1>(oldest) << " лет\n";

    // С самым высоким баллом
    auto best = *std::max_element(students.begin(), students.end(),
        [](const auto& a, const auto& b) {
            return std::get<2>(a) < std::get<2>(b);
        });
    std::cout << "Лучший студент: " << std::get<0>(best)
              << ", балл " << std::get<2>(best) << "\n";

    // Средний возраст
    int totalAge = std::accumulate(students.begin(), students.end(), 0,
        [](int sum, const auto& s) {
            return sum + std::get<1>(s);
        });
    std::cout << std::fixed << std::setprecision(1);
    std::cout << "Средний возраст: "
              << static_cast<double>(totalAge) / students.size() << "\n";

    // Сортировка по баллу
    std::sort(students.begin(), students.end(),
        [](const auto& a, const auto& b) {
            return std::get<2>(a) > std::get<2>(b);
        });

    std::cout << "\nПо баллу:\n";
    for (const auto& [name, age, gpa] : students) {
        std::cout << name << ": " << gpa << "\n";
    }

    return 0;
}
```

Задание 5: Объединение и пересечение списков

Задача: Даны два списка. Найти:

- Объединение (все элементы без дубликатов)
- Пересечение (общие элементы)
- Разность (элементы первого, которых нет во втором)

```python
list1 = [1, 2, 3, 4, 5]
list2 = [4, 5, 6, 7, 8]

# Через множества
set1 = set(list1)
set2 = set(list2)

union = set1 | set2
intersection = set1 & set2
difference = set1 - set2

print(f"Объединение: {sorted(union)}")        # [1, 2, 3, 4, 5, 6, 7, 8]
print(f"Пересечение: {sorted(intersection)}") # [4, 5]
print(f"Разность: {sorted(difference)}")      # [1, 2, 3]

# С сохранением порядка (через list comprehension)
union_ordered = list1 + [x for x in list2 if x not in list1]
intersection_ordered = [x for x in list1 if x in list2]
```

```cpp
#include <iostream>
#include <vector>
#include <set>
#include <algorithm>
#include <iterator>

int main() {
    std::vector<int> list1 = {1, 2, 3, 4, 5};
    std::vector<int> list2 = {4, 5, 6, 7, 8};

    // Через множества
    std::set<int> set1(list1.begin(), list1.end());
    std::set<int> set2(list2.begin(), list2.end());

    // Объединение
    std::set<int> unionSet;
    std::set_union(set1.begin(), set1.end(),
                   set2.begin(), set2.end(),
                   std::inserter(unionSet, unionSet.begin()));

    // Пересечение
    std::set<int> intersection;
    std::set_intersection(set1.begin(), set1.end(),
                          set2.begin(), set2.end(),
                          std::inserter(intersection, intersection.begin()));

    // Разность
    std::set<int> difference;
    std::set_difference(set1.begin(), set1.end(),
                        set2.begin(), set2.end(),
                        std::inserter(difference, difference.begin()));

    std::cout << "Объединение: ";
    for (int x : unionSet) std::cout << x << " ";
    std::cout << "\n";

    std::cout << "Пересечение: ";
    for (int x : intersection) std::cout << x << " ";
    std::cout << "\n";

    std::cout << "Разность: ";
    for (int x : difference) std::cout << x << " ";
    std::cout << "\n";

    return 0;
}
```

Задание 6. Работа в мини-группах

Формат работы: группы по 3-4 человека.
Задача: Каждая группа получает свой вариант. Необходимо реализовать программу на **двух языках** (Python и C++) и сравнить результаты.

Варианты задач для групп:

Вариант	Задача
1	Найти второй по величине элемент в списке
2	Проверить, является ли список отсортированным
3	Найти все пары чисел, сумма которых равна заданному числу
4	Сдвинуть список на K позиций влево/вправо
5	Найти медиану списка
6	Реализовать стек на списке (push, pop)
7	Реализовать очередь на списке (enqueue, dequeue)
8	Найти самый частый элемент в списке

Критерии оценки:

- Корректность решения на двух языках
- Понимание различий между языками
- Обработка краевых случаев
- Качество форматирования вывода

Презентация результатов: 2–3 минуты на группу.

Требования к отчёту и критерии оценки

Структура отчёта:

1. Титульный лист
2. Цель работы
3. Выполнение заданий:
   - Листинг кода для каждого задания
   - Скриншоты результатов
   - Примеры работы со списками, кортежами, векторами
   - Результаты групповой работы
4. Ответы на контрольные вопросы
5. Вывод

Критерии оценки:

Критерий	Мин. балл	Макс. балл
Самостоятельная проработка теории, ответы на вопросы	1	2
Корректность выполнения заданий, рабочий код	3	6
Качество кода (PEP8, структура, комментарии)	1	2
ИТОГО	**5**	**10**

Формат сдачи: отчёт в формате PDF или DOCX + код в Git-репозитории (не менее 3 коммитов).

Контрольные вопросы

Контрольные вопросы

Вопросы для самопроверки:

1. **Что такое коллекция?** Какие виды коллекций вы знаете в Python и C++?
2. **В чём разница между списком и кортежем в Python?** Когда что использовать?
3. **Что такое срез?** Как работает numbers[2:5]? А numbers[::-1]?
4. **Как в C++ создать динамический массив?** Какие преимущества у std::vector?
5. **В чём разница между** int arr[5] **и** std::array<int, 5>**?**
6. **Как отсортировать список в Python?** А вектор в C++?
7. **Как найти пересечение двух списков?** Приведите пример на двух языках.
8. **Что такое многомерная коллекция?** Как создать матрицу?



Ответы должны быть в отчёте.

Задания для самостоятельной работы

Задания для самостоятельной работы

Дополнительные задания (для продвинутых):

1. **Сортировка слиянием:** Реализовать алгоритм сортировки слиянием для списка.

2. **Быстрая сортировка:** Реализовать алгоритм quicksort.

3. **Бинарный поиск:** Реализовать алгоритм бинарного поиска в отсортированном списке.

4. **Матричные операции**: Реализовать сложение, вычитание и умножение матриц.

5. **Сравнение языков (*):** Напишите одинаковую программу на Python и C++ и сравните:
   - Количество строк кода
   - Читаемость
   - Время выполнения
   - Удобство работы с коллекциями.



Дополнительно:
- Создать README.md в репозитории

Спасибо
за внимание

